

FreshCall: sizes tomorrow's fresh order for the store's own costs

Repository: github.com/harry00001177/FreshCall

Data: kaggle.com/competitions/favorita-grocery-sales-forecasting

1. The problem, and what changes

The manager of a fresh-food business, such as a supermarket's fresh section or a fast-food restaurant, orders about 40 perishable items every evening, usually by copying what sold on the same day last week. That number carries no indication of how far it can be trusted, and over-ordering ends as waste. FreshCall gives every item an order in whole cases, a likely range and a one-line reason, lists the least certain items first, and leaves every line for the manager to confirm or change.

I evaluated it on Corporación Favorita supermarket sales (125 million rows), using four stores of about 430 perishable items and three monthly test windows. Each design was confirmed once on a store not used during development, against success criteria committed to git beforehand. On the items that need judgement, wrong orders fell from 13.3 to 10.3 per 40-item night, with 11% fewer units wasted and 26% fewer units short. Ordering for the store's own costs was 21 to 35% cheaper than copying last week at cost ratios of 2 and above.

2. Why AI, and which kind

The task is a bounded numeric prediction with years of history, which suits narrow machine learning. I used quantile gradient-boosted regression because it predicts a range of demand that can be checked for calibration, which an LLM cannot provide. I rejected RAG (no document corpus) and an agent (the task is a single pass). The non-AI baseline is the manager's current practice, scored on the same item-days.

An LLM originally wrote each line's reason. I removed it after the evaluation showed that it used intensity words such as "significantly" with nothing checking the size of the gap, and that it did not reliably apply a numeric rule ("about the same" within 10%). A fixed template now writes the reason. The LLM (Claude Haiku 4.5) remains only as an evaluation judge, where its errors meet a human label rather than a manager.

3. Own or rent, and what it costs

Layer	Own / rent	Choice and reason
Interface	Own	Streamlit, one page, no LLM call
Orchestration	Own	Plain Python; a single pass needs no agent or framework
Demand model	Own	scikit-learn; the ordering logic is specific to this problem
Language model	Rent	Claude Haiku 4.5 via OpenRouter, evaluation only
Evaluation	Own	pytest (156 tests) and one script per experiment

The manager's screen makes no LLM call, so the cost to serve is close to zero, and all LLM work in the project cost at most USD 1.11. I did not use low-code tools because the ordering rule and its thresholds had to be unit-tested and version-controlled for each pre-registered test to be checkable.

4. Trade-offs that shaped the system

Abstaining or answering every item. The original design handed uncertain items back to the manager. Technically it partly worked, as the second gate picked risky items 1.26 times better than random. In business terms it failed, because it handed back about 75% of items, and even on those items the model was wrong less often than last week's number (39.6% against 46.5%). A gate only adds value if the person receiving the item knows something the model does not, and this data cannot show that. I therefore gave every item an order and expressed uncertainty as a range, which keeps the manager in control without asking for work the data cannot justify.

A fixed service level or a store setting. Rounding a median forecast up to whole cases already covered demand on 85 to 91% of days, so the default order was a high-service policy that no one had chosen. I made the balance between waste and stockout an explicit cost ratio and order the case count with the lowest expected cost. This returns a business decision to the business, at the price of one more setting. A related rule shows the same trade-off at smaller scale. Guaranteeing one case for items that sold every day last week costs almost nothing at ratio 4 (cost 420.3 to 420.6), but at ratio 0.25 it raises cost from 99.2 to 157.6, so it is reasonable only where running out is the larger worry.

Fluent text or checkable text. The LLM sentence read better, and the template is plainer. I chose the template because every word and number in it can be tested, and because a fluent sentence that misleads is worse than a plain one that does not.

Evaluation rigour or speed. Pre-registration with one run per confirmation store made results harder to bias, but each design used up a store. Three stores went to confirmation, and all four stores used are in Quito.

5. Critique of the outcomes

Metrics. Case Match Rate counts cases only, and a correct case count can still waste five units, so I added units wasted and short. Both are distorted by the missing stock data. Each day starts from zero, so a 12-unit case for an item selling three a day counts nine units as waste. This is also why low cost ratios look attractive, since most of their saving comes from ordering nothing (up to 64% of days). The original 15% improvement target was not met (+4% to +12% on the development store), and excluding discontinued items raises it to +14.6%, which shows how sensitive the figure is to the item set.

Evaluations. The manager is simulated as accepting every suggestion, and the minutes rely on unmeasured timings, reported as a grid of nine assumptions. The sentence checks used 6 to 17 sentences with me as the only labeller. The judge was wrong on "direction" in 7 of 14 sentences, so it serves only as a first screen. The test months exclude the December peak.

Difficulties and tuning. The first full backtest was far worse than the pilot; I misdiagnosed the cause, recorded the correction, and pre-registered the fix before re-running. The first newsvendor rule lost at ratios 2 and 4 on the development stores because it targeted a high quantile and then rounded up again. I amended it and registered it again before touching store 45. A deliberate leak raised the improvement from +11.8% to +20.7%, a false pass of the target that the unit tests catch and calibration does not. I also found and corrected a timing error in my own value table (47.5 to about 16 minutes), keeping both versions in the log.

Rough edges. The model's hyperparameters were never tuned, one case size of 12 is assumed for every item, slow items show more waste than last week's number, and one-off events such as a local promotion are not detected.

6. Risks and guardrails

Risk	Guardrail	Status
LLM invents a number	Numeral check on any LLM text; template fallback	Built
A true sentence that misleads	LLM removed from the manager's screen	Built
Silent drift	Bias and coverage monitors; a simulated 50% demand jump raised an alert the next day	Built
Over-trust	No confidence score; every line editable and logged	Built
Unseen events (promotion, closure)	Novelty check, calendar rule	Not built
Stockouts recorded as low demand	Needs stock data	Not possible here

FreshCall is advisory. A person confirms every order, and it is not intended for food-safety decisions such as discard times. In line with Singapore's IMDA Model AI Governance Framework, a human stays in the loop for this low-risk, high-frequency decision. Of the OWASP Top 10 for LLM Applications (2025), the risk addressed most directly is misinformation.

7. Future path

The main next step is stock data. Public datasets with daily on-hand inventory would allow ordering that carries leftovers to the next day to be tested, and stockout labels would stop sold-out days from being learned as low demand. A store trial could then measure ordering time and how often managers change suggestions. An LLM could later return to the screen to turn a manager's note, such as "school holiday tomorrow", into an adjustment that passes the same numeral check.

The code was written with AI assistance (Claude Code). Every design decision and every human label is mine, and every number above was run and can be reproduced from the repository.